Today

Welcome to your next lesson: Object identity vs equality.

I'm setting up your lesson based on what you've learned so far, the next steps for you, and how you learn best.

Step by step, you're moving forward.

Imagine you're at a pet shelter 🐶

There's one actual dog in front of you, but it has two name tags on its collar:

one says "Luna" and another says "Favorite".

It's still one real dog (one identity).
But you have two names for it.

In Python, this is like two variables pointing to the same object.

Quick gut check:

If luna and favorite_pet are just two names for the same pet object,

and we change favorite_pet.age,

do you expect luna.age to change too?

Just answer: yes or no.

Yes

Exactly — yes ✅

When two variables point to the same object, any change through one name is visible through the other. That's the idea of object identity.

Here's a visual for that shelter example 👇

Now, let's connect this to code step by step.

In your Python editor, set up this tiny example:

Python
```
class Pet:
    pass

luna = Pet()
favorite_pet = luna

print(luna is favorite_pet)
```
Don't add anything else yet.

Type this into the editor.
Run it (no need to submit yet).
Tell me what it printed: True or False?
Two names, one pet object

True

Nice, that True tells us: luna and favorite_pet are two names for the same object 🎯

In the diagram below, both arrows from luna and favorite_pet go to the same Pet box — that's shared identity (aliasing).

Your turn, just to check understanding:

Looking at the diagram, how would you describe it in your own words?

One short sentence like:

“Both names point to the same pet object.”

Write it in your own words, then we’ll make that pet change its state and watch what happens 👀

Two names, one pet object

both names point to the same instance of the pet object, theres only one object in memory and both variables point to it

Perfectly said 💯

You’ve got object identity here: one Pet object in memory, two variables (luna, favorite_pet) both pointing to it.

Let’s see the shared state part now.

In your editor, change the code to this:

Python

```
class Pet:
    pass

luna = Pet()
favorite_pet = luna

luna.name = "Luna"
favorite_pet.age = 3

print(luna.name)
print(luna.age)
```

Run it (no need to submit yet).
Tell me: what output did you get for those two print lines?
Luna
3

Great — that matches what I’d expect ✅

Key idea here:

luna and favorite_pet are the same object.
So when you set favorite_pet.age = 3, that age lives inside that one object.
When you read luna.age, Python looks in the same object and finds 3.

So: change through one name, visible through the other → that's shared state via identity.

Next step: let's contrast this with two separate pets that just happen to have the same data (same name, same age).

Before we touch code:

If I make:

pet1 = a Pet named "Luna", age 3
pet2 = another Pet also named "Luna", age 3

Do you think pet1 and pet2 are:

the same object,
or two different objects with equal values?

Just answer: "same object" or "different objects with equal values".

different

Good instinct: they're different objects with equal values 🎯

So now we have two ideas:

Same object (identity): two names, one pet in memory.
Different objects, same values (equality): two pets that just happen to have the same name/age.
Let's make that concrete in code now.

In your editor, replace your code with this:

```
Python
class Pet:
    pass

pet1 = Pet()
pet1.name = "Luna"
pet1.age = 3

pet2 = Pet()
pet2.name = "Luna"
pet2.age = 3

print(pet1 is pet2)
print(pet1.name == pet2.name)
print(pet1.age == pet2.age)
Run (no need to submit yet), then tell me the three outputs you see, in order.

False
True
True

Nice work ✅


So from that output:


pet1 is pet2 → False → they are not the same object (different identities).
pet1.name == pet2.name → True → the names are equal.
pet1.age == pet2.age → True → the ages are equal.

So: different objects, equal attribute values.

That’s the difference between identity (is) and equality (== on values).

Quick check in your own words (short answer is fine):


If you see a is b is False, but some print(a.x == b.x) are True, what does that tell you about a and b?

They’re different objects?
```

Exactly 👍

If a is b is False but some a.x == b.x are True, that means:

a and b are different objects (different identity)
but some of their values/attributes are equal

So: not the same thing in memory, but matching data.

Now let’s see why this matters when state changes.

Keep your last code, and just add these two lines at the end:

Python
pet1.age = 4

print(pet1.age)
print(pet2.age)
Run it, and tell me the last two lines of output you see.

4
3

Perfect 👌

Those last two lines:

pet1.age → 4
pet2.age → 3

This shows:

They are separate objects. Changing pet1.age does not affect pet2.age.
So different identity → no shared state → each keeps its own age.

Compare that to our earlier luna / favorite_pet case, where changing one name changed what we saw through the other.

Let’s pause and lock this in with one simple question:

When you’re asking “are these two variables pointing to the same object?”, which operator should you use in Python: is or ==?

is

What do you think? 👨‍🔬

Pick your answer in the question box above 👆

Two pet variables point to the same object. Which operator wins the identity detective badge?

A.
is

B.
==

C.
=

Nice job, Ron — you nailed it 👨‍🎉

is wins the identity detective badge because it answers:
“Do these two variables point to the same object in memory?”

== is for value equality:
“Do these two values match?”

= just assigns a value to a variable, it doesn’t compare anything.

You’ve shown you can:

Tell the difference between same object vs same values
Explain how changing one alias affects shared state, but separate objects keep their own state

That’s exactly what this lesson was about.

If you want, you can keep experimenting with your Pet class, or move on to the next lesson when you’re ready 🚀